

Experiment No. 1

Perform tokenization (Whitespace, Punctuation-based, Treebank, Tweet, MWE) using NLTK library. Use porter stemmer and snowball stemmer for stemming. Use any technique for lemmatization

Install & Import Libraries

```
import nltk
from nltk.tokenize import word_tokenize, WhitespaceTokenizer, TreebankWordTokenizer, TweetTokenizer, MWETokenizer
from nltk.stem import PorterStemmer, SnowballStemmer, WordNetLemmatizer
from nltk.corpus import wordnet

# Download required datasets
nltk.download('punkt')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('punkt_tab')
```

```
[nltk_data] Downloading package punkt to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt.zip.
[nltk_data] Downloading package wordnet to /root/nltk_data...
[nltk_data] Downloading package omw-1.4 to /root/nltk_data...
[nltk_data] Downloading package punkt_tab to /root/nltk_data...
[nltk_data] Unzipping tokenizers/punkt_tab.zip.
True
```

```
text = "Natural Language Processing is really amazing! I am learning some concepts like tokenization, stemming, and lemmatization."
print("Original Text:\n", text)
```

```
Original Text:
Natural Language Processing is really amazing! I am learning some concepts like tokenization, stemming, and lemmatization.
```

Tokenization Techniques

(A) Whitespace Tokenization

```
ws_tokenizer = WhitespaceTokenizer().tokenize(text)
print("Whitespace Tokens:\n", ws_tokenizer)
```

```
Whitespace Tokens:
['Natural', 'Language', 'Processing', 'is', 'really', 'amazing!', 'I', 'am', 'learning', 'some', 'concepts', 'like', 'tokenization', 'stemming', 'and', 'lemmatization']
```

(B) Punctuation-based Tokenization (word_tokenize)

```
punct_tokens = word_tokenize(text)
print("Punctuation based Tokens:\n", punct_tokens)
```

```
Punctuation based Tokens:
['Natural', 'Language', 'Processing', 'is', 'really', 'amazing', '!', 'I', 'am', 'learning', 'some', 'concepts', 'like', 'tokenization', 'stemming', 'and', 'lemmatization']
```

(C) Treebank Tokenization

```
treebank_tokenizer = TreebankWordTokenizer().tokenize(text)
print("Treebank Tokens:\n", treebank_tokenizer)
```

```
Treebank Tokens:
['Natural', 'Language', 'Processing', 'is', 'really', 'amazing', '!', 'I', 'am', 'learning', 'some', 'concepts', 'like', 'tokenization', 'stemming', 'and', 'lemmatization']
```

(D) Tweet Tokenization

```
tweet_tokenizer = TweetTokenizer().tokenize(text)
print("Tweet Tokens:\n", tweet_tokenizer)
```

```
Tweet Tokens:
['Natural', 'Language', 'Processing', 'is', 'really', 'amazing', '!', 'I', 'am', 'learning', 'some', 'concepts', 'like', 'tokenization', 'stemming', 'and', 'lemmatization']
```

(E) MWE Tokenization (Multi-Word Expression)

```
mwe_tokenizer = MWETokenizer([('New', 'York'), ('machine', 'learning')])
sample_text = "I love machine learning and visiting New York."
mwe_tokens = mwe_tokenizer.tokenize(sample_text.split())
print("MWE Tokens:\n", mwe_tokens)
```

```
MWE Tokens:
['I', 'love', 'machine_learning', 'and', 'visiting', 'New', 'York.']
```

✓ Stemming

(A) Porter Stemmer

```
porter = PorterStemmer()
words = word_tokenize(text)

porter_stems = [porter.stem(word) for word in words]
print("Porter Stemmer Output:\n", porter_stems)
```

```
Porter Stemmer Output:
['natur', 'languag', 'process', 'is', 'realli', 'amaz', '!', 'i', 'am', 'learn', 'some', 'concept', 'like', 'token', ',', ',']
```

(B) Snowball Stemmer

```
snowball = SnowballStemmer("english")

snowball_stems = [snowball.stem(word) for word in words]
print("Snowball Stemmer Output:\n", snowball_stems)
```

```
Snowball Stemmer Output:
['natur', 'languag', 'process', 'is', 'realli', 'amaz', '!', 'i', 'am', 'learn', 'some', 'concept', 'like', 'token', ',', ',']
```

✓ Lemmatization (WordNet Lemmatizer)

```
lemmatizer = WordNetLemmatizer()

lemmatized_words = [lemmatizer.lemmatize(word) for word in words]
print("Lemmatized Words:\n", lemmatized_words)
```

```
Lemmatized Words:
['Natural', 'Language', 'Processing', 'is', 'really', 'amazing', '!', 'I', 'am', 'learning', 'some', 'concept', 'like', 'to']
```

Start coding or [generate](#) with AI.